

ABSTRACT

The Online Shopping System is a database-based e-commerce application that helps customers buy products online easily and efficiently. Customers can browse products, add items to a cart, place orders, make payments, track deliveries, and give reviews. The main problem with traditional manual or spreadsheet-based systems is that they create duplicate records, are difficult to manage, and become slow as the number of customers and products increases.

The purpose of this database is to store and manage all shopping-related information in one organized system. It includes details about customers, products, categories, orders, payments, shipping, and reviews. The database ensures that data is accurate, secure, and easy to access.

The major functionalities of the system include product management, customer registration, shopping cart management, order processing, payment handling, delivery tracking, and review management. The database is designed using a normalized relational structure to reduce redundancy and improve performance.

The key outcome of this project is a reliable, scalable, and efficient database design that supports smooth online shopping operations, maintains data consistency, and provides a strong foundation for future application development.

The database is designed using normalization techniques up to Third Normal Form (3NF) to reduce data redundancy and maintain consistency. Primary keys, foreign keys, unique constraints, and validation rules are implemented to ensure data integrity and accuracy. SQL operations such as data insertion, retrieval, updating, deletion, joins, aggregate functions, views, and subqueries are used to support efficient database management and reporting.

The system provides a centralized repository for managing online shopping transactions, enabling quick access to information and effective handling of customer orders. By maintaining accurate and structured data, the Online Shopping System improves operational efficiency, minimizes errors, and supports reliable decision-making. This project demonstrates the practical application of database management concepts in developing a scalable and well-structured e-commerce database solution.

INDEX

Chapter	Title	Page No.
	Certificate	1
	Declaration	2
	Acknowledgement	3
	Abstract	4
	List of Figures	5
	List of Tables	6
1	Introduction	5
	1.1 Introduction to the Domain	5
	1.2 Problem Statement	5
	1.3 Motivation and Need for a Database	6
	1.4 Objectives of the Project	6
	1.5 Scope of the Project	6
	1.6 Organization of the Report	7
2	System Study and Tools	8
	2.1 Existing System and Its Limitations	8
	2.2 Proposed Database System	8
	2.3 Tools and Technologies Used	8
	2.4 Software and Hardware Requirements	9
3	Database Design (Review – I)	10
	3.1 Identified Entities	10
	3.2 Entity Attributes and Attribute Types	10
	3.3 ER Diagram	12
	3.4 Relationships and Cardinality	12
	3.5 Relational Database Schema	13
	3.6 Primary Keys and Foreign Keys	16
	3.7 Normalization up to 3NF	17
4	Database Implementation (Review – II)	18
	4.1 Database and Table Creation – DDL	18
	4.2 Sample Data Insertion – DML	21
	4.3 Constraints and Data Integrity	26
	4.4 Data Validation Rules	27
5	SQL Operations and Queries	29
	5.1 DQL – Basic Retrieval Queries	29
	5.2 Join Operations	30
	5.3 Aggregate Functions, GROUP BY and HAVING	32
	5.4 Nested and Correlated Subqueries	33
	5.5 Views and Relational Set Operations	35
	5.6 DCL and TCL Operations	37
6	Modules and Functionalities	38

7	Results – Sample Outputs and Screenshots	39
8	Testing and Data Validation	42
9	GitHub Repository and Team Contribution	43
10	Conclusion and Future Enhancements	45
	References	
	Appendix A – Complete SQL Script	



LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
Fig. 3.1	ER Diagram of the online shopping System	12
Fig. 3.2	Relational Schema Diagram	16
Fig. 4.1	Successful Creation of Tables	21
Fig. 4.2	Sample Data Inserted into the Tables	23
Fig. 4.3	Referential Integrity Enforced by the Database	27
Fig. 7.1	Shows customer order history output	39
Fig. 7.2	Product inventory output	40

LIST OF TABLES

Table No.	Title of the Table	Page No.
Table 2.1	Tools and Technologies Used	8
Table 2.2	Software and Hardware Requirements	9
Table 3.1	Entities Identified for the Selected Domain	10
Table 3.2	Attributes of Each Entity	10
Table 3.3	Relationships and Cardinality	12
Table 3.4	Relational Database Schema	13
Table 3.5	Primary Key and Foreign Key Definitions	16
Table 3.6	Normalization Steps	17
Table 4.1	Sample Data Summary	21
Table 4.2	Constraints and Data Integrity Rules	26
Table 6.1	Modules and Their Functionalities	38
Table 8.1	Test Cases and Results	42
Table 9.1	Team Responsibilities	43
Table 9.2	Review-wise Progress Record	43

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Domain

The Online Shopping System is an e-commerce application that allows customers to purchase products through the internet. It provides a convenient platform where users can browse products, add items to a shopping cart, place orders, make online payments, track deliveries, and submit reviews. The system also helps administrators manage products, inventory, orders, payments, and customer information efficiently. As online shopping continues to grow, businesses need a reliable way to store and manage large amounts of data generated by customers and transactions.

A database is required because the system handles thousands of products, customers, orders, and payments that must be stored accurately without duplication. It ensures data integrity, security, and consistency while supporting multiple users at the same time. The database also enables fast retrieval of information, efficient order processing, inventory management, and report generation. Unlike manual records or spreadsheets, a database provides structured storage, transaction management, and scalability, making the online shopping system more reliable and efficient.

1.2 Problem Statement

Existing problem:

The Online Shopping System aims to overcome the limitations of traditional file-based retail management systems, such as data redundancy, integrity issues, poor scalability, weak security, and inefficient data retrieval. The proposed solution is a normalized relational database that manages customers, products, orders, payments, shipping, and reviews through interconnected tables with integrity constraints. This database ensures accurate, secure, scalable, and efficient management of e-commerce operations while supporting reliable transaction processing and reporting.

Limitations:

Traditional retail businesses and small online sellers often manage customer, product, order, and payment information using spreadsheets or manual record-keeping methods. As the volume of transactions increases, these approaches become inefficient and difficult to maintain. Customer details, product information, and order records may be duplicated across multiple files, leading to data redundancy and inconsistency. Retrieving information such as order history, product availability, and sales reports becomes time-consuming and prone to errors.

In addition, manual systems lack proper integrity checks and security mechanisms. Orders may reference unavailable products, inventory levels may not be updated accurately, and unauthorized access to sensitive customer or payment information may occur. The absence of a centralized database also makes it difficult to maintain data consistency, support concurrent users, and generate meaningful business reports.

To address these challenges, this project proposes an Online Shopping System database that stores and manages customers, products, categories, orders, payments, shipping details, and reviews in a structured relational database. The system enforces data integrity through primary keys, foreign keys, and validation constraints, reduces redundancy through normalization, and enables efficient data retrieval using SQL queries. This provides a reliable, secure, and scalable solution for managing e-commerce operations effectively.

1.3 Motivation and Need for a Database

The rapid growth of e-commerce platforms has increased the volume of data generated through customer registrations, product listings, orders, payments, shipping details, and product reviews. Managing such large amounts of information using manual records, spreadsheets, or separate files often leads to data redundancy, inconsistency, and difficulties in data retrieval and maintenance.

An Online Shopping System requires a Database Management System (DBMS) to store and manage data in a structured and organized manner. A DBMS ensures data integrity through primary keys, foreign keys, and validation constraints, thereby preventing duplicate and invalid records. It also supports efficient retrieval of information through SQL queries, enabling quick access to customer details, product inventories, order histories, and sales reports.

Furthermore, a database provides multi-user access, transaction management, security, and scalability, which are essential for modern e-commerce applications. As the number of customers and products increases, the database can efficiently handle large volumes of data while maintaining consistency and performance. Therefore, implementing a relational database is essential for ensuring reliable, secure, and efficient management of the Online Shopping System.

1.4 Objectives of the Project

1. To design a normalized relational database for the online shopping system with at least six entities, and to represent the design as an ER diagram showing all attributes, relationships and cardinalities
2. To implement the database with appropriate primary keys, foreign keys and integrity constraints.
3. To perform SQL operations including DDL, DML, DQL, joins, aggregate functions, subqueries, views, and set operations for effective data management and retrieval.
4. To enforce data integrity and validation through constraints such as NOT NULL, UNIQUE, CHECK, DEFAULT, and referential integrity rules.
5. To test the database using transaction control operations, security commands, and sample business queries to ensure correctness, consistency, and reliability.

1.5 Scope of the Project

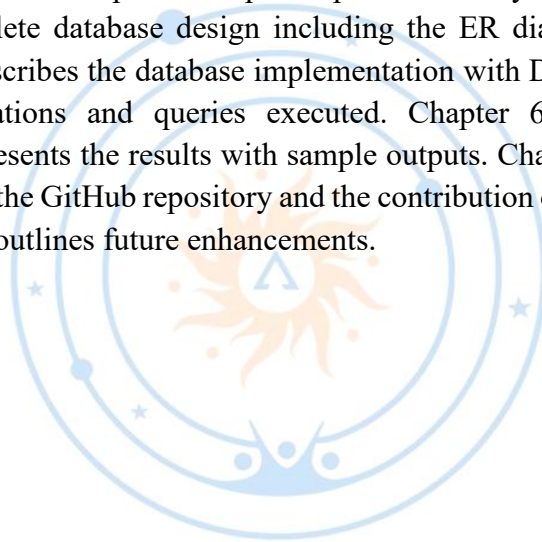
The Online Shopping System database covers the storage and management of customer information, product categories, products, shopping carts, orders, order items, payments, shipping details, and

customer reviews. It supports essential e-commerce operations such as customer registration, product browsing, cart management, order placement, payment recording, shipment tracking, and review management. The database also provides facilities for maintaining data integrity, enforcing relationships between entities, and generating reports through SQL queries. The design is normalized up to Third Normal Form (3NF) to minimize redundancy and ensure consistency of data.

The database does not cover the development of a complete web or mobile application interface, real-time payment gateway integration, live inventory synchronization with external vendors, product recommendation systems, online customer support, advanced security features such as encryption and multi-factor authentication, cloud deployment, backup and recovery mechanisms, or business analytics dashboards. These features are beyond the scope of the current project and may be considered for future enhancements.

1.6 Organization of the Report

This report is organized into ten chapters. Chapter 2 presents the system study and the tools used. Chapter 3 covers the complete database design including the ER diagram, relational schema and normalization. Chapter 4 describes the database implementation with DDL and sample data. Chapter 5 presents the SQL operations and queries executed. Chapter 6 explains the modules and functionalities. Chapter 7 presents the results with sample outputs. Chapter 8 covers testing and data validation. Chapter 9 details the GitHub repository and the contribution of each team member. Chapter 10 concludes the report and outlines future enhancements.



CHAPTER 2: SYSTEM STUDY AND TOOLS

2.1 Existing System and Its Limitations

The existing system for managing online shopping activities often relies on manual record keeping, spreadsheets, or independent files to store customer, product, and order information. As the number of customers and transactions increases, maintaining data becomes difficult and time-consuming. Data redundancy, inconsistency, and retrieval delays are common problems. Additionally, manual systems do not provide proper data integrity, security, or support for multiple users accessing data simultaneously.

Limitations of the Existing System

- Data redundancy and duplication.
- Difficulty in maintaining large volumes of data.
- Slow retrieval of customer and order information.
- Poor data integrity and consistency.
- Limited security and access control.
- Difficulty in generating reports and summaries.

2.2 Proposed Database System

The proposed Online Shopping System uses a relational database to store and manage customer, product, order, payment, shipping, and review information in a structured manner. The system minimizes redundancy through normalization and maintains data integrity using primary keys, foreign keys, and validation constraints. It enables efficient storage, retrieval, and management of e-commerce data through SQL operations.

Advantages of the Proposed System

- Centralized and organized data storage.
- Reduced data redundancy and inconsistency.
- Faster data retrieval using SQL queries.
- Improved security and integrity through constraints.
- Support for multiple related entities and transactions.
- Easy generation of reports and summaries.

2.3 Tools and Technologies Used

Table 2.1: Tools and Technologies Used

Category	Tool / Technology Used	Version / Remarks
Database Server	MySQL	MySQL 8.0
Query / Administration Tool	MySQL Workbench	Used for creating tables and executing SQL queries

ER Diagram Tool	draw.io (diagrams.net)	Used for ER Diagram and Relational Schema Design
Version Control	GitHub	Repository for project files and collaboration
Documentation	Microsoft Word	Used for report preparation and documentation

2.4 Software and Hardware Requirements

Table 2.2: Software and Hardware Requirements

Requirement Type	Specification
Operating System	Windows 10 / Windows 11
Database Server	MySQL 8.0
Query / Administration Tool	MySQL Workbench
ER Diagram Tool	draw.io (diagrams.net)
Version Control	GitHub
Documentation Tool	Microsoft Word
Processor	Intel Core i3 / AMD Ryzen 3 or higher
RAM	Minimum 4 GB (Recommended 8 GB)
Storage	Minimum 20 GB Free Disk Space
Internet Connection	Required for GitHub and online resources
Display	1366 × 768 Resolution or higher

CHAPTER 3: DATABASE DESIGN

3.1 Identified Entities

Six core entities were identified from the study of the domain, together with two supporting tables introduced during normalization to hold multi-valued attributes.

Table 3.1: Entities Identified for the Selected Domain

S.No	Entity Name	Description
1.	Customer	Registered buyer with login, contact and address details
2.	Product	Item on sale with price, stock and description
3.	Category	Group that classifies products (Electronics, Fashion...)
4.	Order	A purchase placed by a customer on a given date.
5.	Payment	Transaction record for an order: mode, amount, status.
6.	Shipping	Delivery details, courier and tracking for an order.

3.2 Entity Attributes and Attribute Types

The attributes of each entity are listed below with the data type, the key role and the attribute classification. The classification follows the ER model: an attribute is Simple if it cannot be divided further, Composite if it is made up of smaller attributes, Multi-valued if one entity instance may hold several values, and Derived if its value is computed from other stored attributes.

Table 3.2: Attributes of Each Entity

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Customer	customer_id	INT	PK	Simple
Customer	name	VARCHAR(100)	—	Composite
Customer	email	VARCHAR(100)	—	Simple
Customer	phone	VARCHAR(15)	—	Multi-valued
Customer	password	VARCHAR(255)	—	Simple
Customer	address	VARCHAR(255)	—	Composite
Customer	city	VARCHAR(50)	—	Simple
Customer	pincode	VARCHAR(10)	—	Simple
Product	product_id	INT	PK	Simple
Product	name	VARCHAR(100)	—	Simple
Product	description	TEXT	—	Simple

Product	price	DECIMAL(10,2)	—	Simple
Product	stock_qty	INT	—	Simple
Product	category_id	INT	FK	Simple
Category	category_id	INT	PK	Simple
Category	category_name	VARCHAR(100)	—	Simple
Category	description	TEXT		Simple
Order	order_id	INT	PK	Simple
Order	order_date	DATE	—	Simple
Order	total_amount	DECIMAL(10,2)	—	Derived
Order	status	VARCHAR(20)	—	Simple
Order	customer_id	INT	FK	Simple
Order_Item	order_id	INT	PK, FK	Simple
Order_Item	product_id	INT	PK, FK	Simple
Order_Item	quantity	INT	—	Simple
Order_Item	unit_price	DECIMAL(10,2)	—	Simple
Payment	payment_id	INT	PK	Simple
Payment	order_id	INT	FK	Simple
Payment	payment_mode	VARCHAR(30)	—	Simple
Payment	amount	DECIMAL(10,2)	—	Simple
Payment	pay_date	DATE	—	Simple
Payment	status	VARCHAR(20)	—	Simple
Shipping	shipping_id	INT	PK	Simple
Shipping	order_id	INT	FK	Simple
Shipping	address	VARCHAR(255)	—	Composite
Shipping	courier	VARCHAR(100)	—	Simple
Shipping	tracking_no	VARCHAR(50)	—	Simple
Shipping	delivery_date	DATE	—	Simple
Review	review_id	INT	PK	Simple
Review	customer_id	INT	FK	Simple
Review	product_id	INT	FK	Simple
Review	rating	INT	—	Simple
Review	comment	TEXT	—	Simple
Review	review_date	DATE	—	Simple

3.3 ER Diagram

The ER diagram below shows the six core entities as rectangles, their attributes as ellipses with primary key attributes underlined, and the relationships as named diamonds with the cardinality marked on every line.

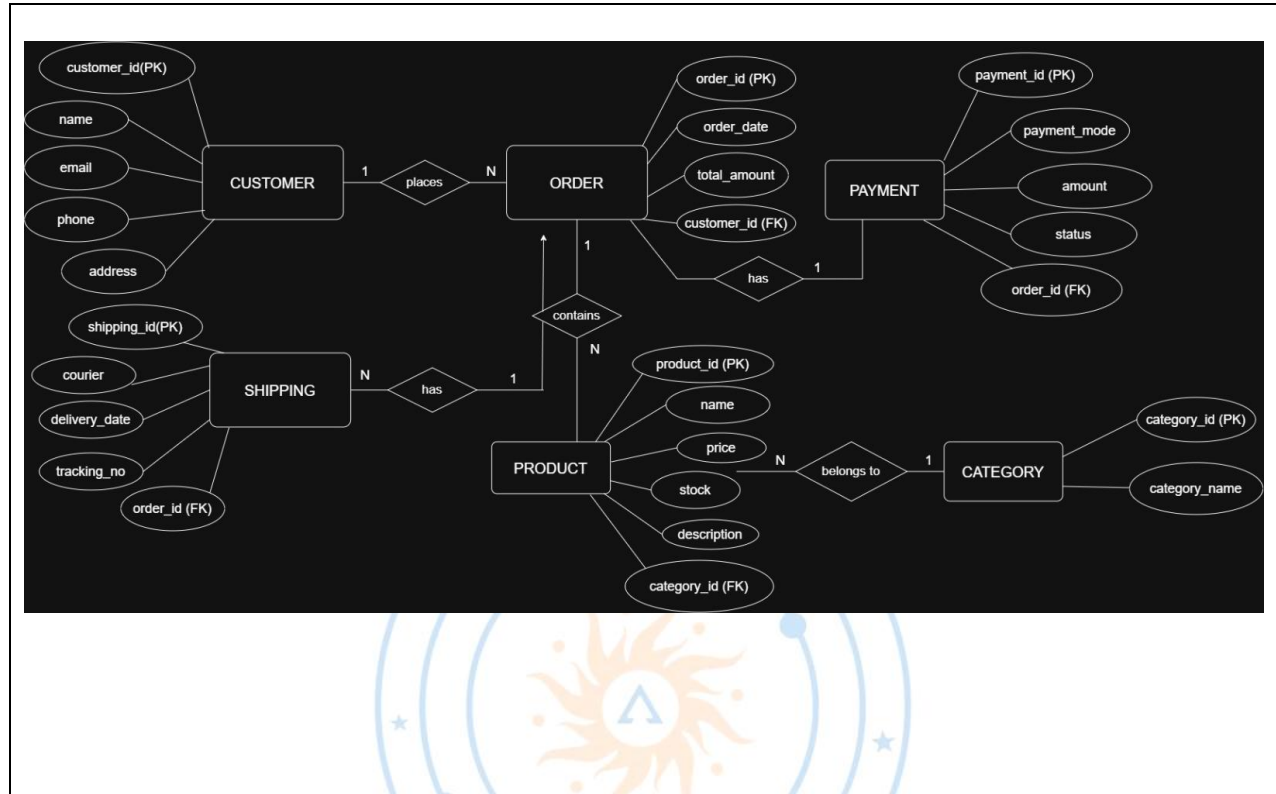


Fig. 3.1: ER Diagram of the Online Shopping System

3.4 Relationships and Cardinality

Table 3.3: Relationships and Cardinality

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)	Participation
CATEGORY	Contains	PRODUCT	1 : M	Partial – Total
CUSTOMER	Places	ORDER	1 : M	Partial – Total
ORDER	Contains	PRODUCT	M : N	Total – Partial
ORDER	Paid By	PAYMENT	1 : 1	Total – Total
ORDER	Shipped Via	SHIPPING	1 : 1	Total – Total
CUSTOMER	Reviews	PRODUCT	M : N	Partial – Partial
CUSTOMER	Owns	CART	1 : 1	Partial – Total

CART	Holds	PRODUCT	M : N	Partial – Partial
------	-------	---------	-------	----------------------

3.5 Relational Database Schema

The ER design was converted into the following relational schema. Each entity became a table, each multi-valued attribute became a separate table, and each relationship was implemented by placing the foreign key on the many side.

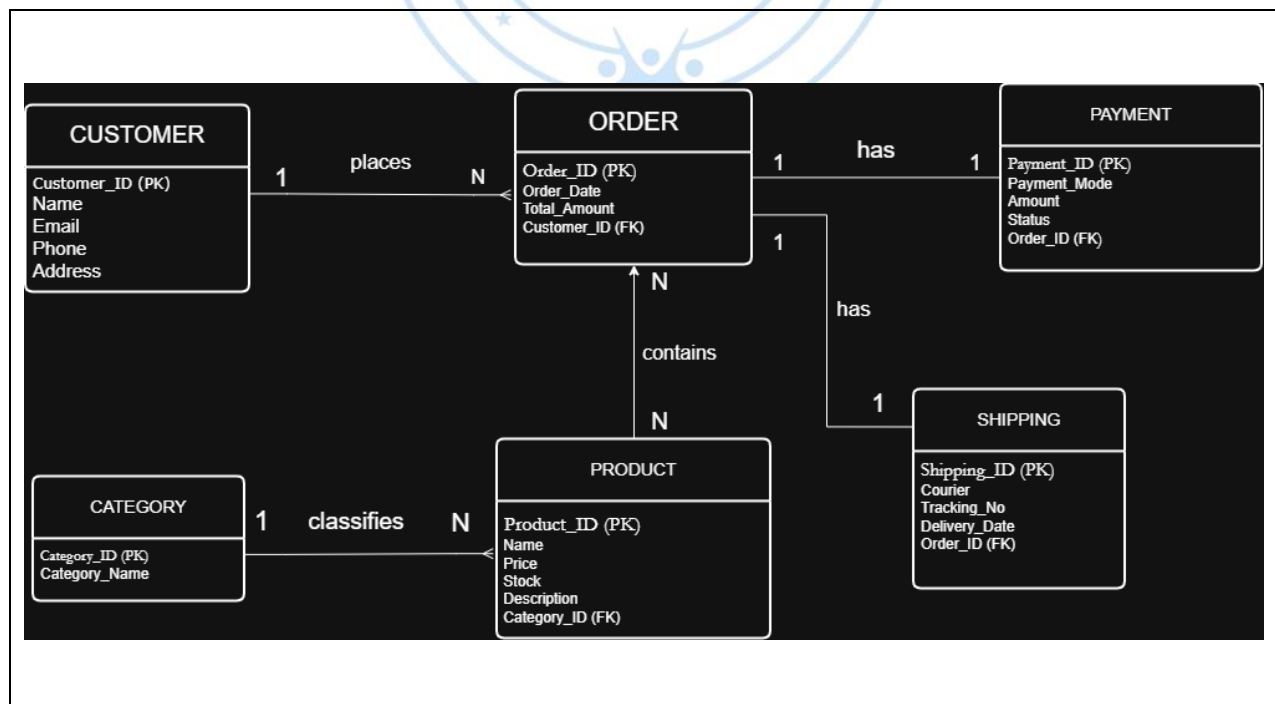
Table 3.4: Relational Database Schema

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Customer	customer_id	INT	PK	NOT NULL
Customer	name	VARCHAR(100)	—	UNIQUE, NOT NULL
Customer	email	VARCHAR(100)	—	NOT NULL
Customer	phone	VARCHAR(15)	—	NOT NULL
Customer	password	VARCHAR(255)	—	NOT NULL
Customer	address	VARCHAR(255)	—	NOT NULL
Customer	city	VARCHAR(50)	—	NOT NULL
Customer	pincode	VARCHAR(10)	—	NOT NULL
Product	product_id	INT	PK	NOT NULL, AUTO_INCREMENT
Product	name	VARCHAR(100)	—	NOT NULL
Product	description	TEXT	—	NULL
Product	price	DECIMAL(10,2)	—	NOT NULL, CHECK(price > 0)
Product	stock_qty	INT	—	DEFAULT 0
Product	category_id	INT	FK	REFERENCES CATEGORY(category_id)
Category	category_id	INT	PK	NOT NULL, AUTO_INCREMENT
Category	category_name	VARCHAR(100)	—	NOT NULL
Category	description	TEXT	—	NULL

Order	order_id	INT	PK	NOT NULL, AUTO_INCREMENT
Order	order_date	DATE	—	NOT NULL
Order	total_amount	DECIMAL(10,2)	—	NOT NULL
Order	status	VARCHAR(20)	—	DEFAULT 'Pending'
Order	customer_id	INT	FK	REFERENCES CUSTOMER(c ustomer_id)
Order_Item	order_id	INT	PK, FK	REFERENCES ORDERS(orde r_id)
Order_Item	product_id	INT	PK, FK	REFERENCES PRODUCT(pr oduct_id)
Order_Item	quantity	INT	—	NOT NULL, CHECK(quant ity > 0)
Order_Item	unit_price	DECIMAL(10,2)	—	NOT NULL
Payment	payment_id	INT	PK	NOT NULL, AUTO_INCREMENT
Payment	order_id	INT	FK	REFERENCES ORDERS(orde r_id)
Payment	payment_mode	VARCHAR(30)	—	NOT NULL
Payment	amount	DECIMAL(10,2)	—	NOT NULL
Payment	pay_date	DATE	—	NOT NULL
Payment	status	VARCHAR(20)	—	DEFAULT 'Success'
Shipping	shipping_id	INT	PK	NOT NULL, AUTO_INCREMENT
Shipping	order_id	INT	FK	REFERENCES ORDERS(orde r_id)
Shipping	address	VARCHAR(255)	—	NOT NULL
Shipping	courier	VARCHAR(100)	—	NOT NULL

Shipping	tracking_no	VARCHAR(50)	—	UNIQUE
Shipping	delivery_date	DATE	—	NULL
Review	review_id	INT	PK	NOT NULL, AUTO_INCREMENT
Review	customer_id	INT	FK	REFERENCES CUSTOMER(c ustomer_id)
Review	product_id	INT	FK	REFERENCES PRODUCT(pr oduct_id)
Review	rating	INT	—	CHECK(rating BETWEEN 1 AND 5)
Review	comment	TEXT	—	NULL
Review	review_date	DATE	—	NOT NULL
CART	cart_id	INT	PK	NOT NULL, AUTO_INCREMENT
CART	customer_id	INT	FK	REFERENCES CUSTOMER(c ustomer_id)
CART	created_date	DATE	—	NOT NULL

Schema Diagram



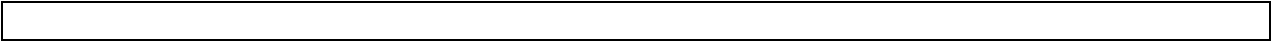


Fig. 3.2: Relational Schema Diagram

3.6 Primary Keys and Foreign Keys

Table 3.5: Primary Key and Foreign Key Definitions

Table	Primary Key	Foreign Key	References (Table.Column)
CUSTOMER	customer_id	—	—
CATEGORY	category_id	—	—
PRODUCT	product_id	category_id	CATEGORY.category_id
ORDER	order_id	customer_id	CUSTOMER.customer_id
ORDER_ITEM	(order_id, product_id)	order_id	ORDER.order_id
ORDER_ITEM	(order_id, product_id)	product_id	PRODUCT.product_id
PAYMENT	payment_id	order_id	ORDER.order_id
SHIPPING	shipping_id	order_id	ORDER.order_id
REVIEW	review_id	customer_id	CUSTOMER.customer_id
REVIEW	review_id	product_id	PRODUCT.product_id
CART	cart_id	customer_id	CUSTOMER.customer_id

3.7 Normalization up to 3NF

Normalization is a database design technique used to organize data efficiently, reduce redundancy, and maintain data integrity. The Online Shopping System database has been normalized up to Third Normal Form (3NF).

First Normal Form (1NF)

A relation is in First Normal Form if all attributes contain atomic values and there are no repeating groups. Initially, customer, product, and order details may be stored in a single table containing multiple product entries for the same order. To achieve 1NF, repeating groups are removed and each attribute is stored as a single value. Separate tables such as CUSTOMER, PRODUCT, ORDERS, and ORDER_ITEM are created to store data in a structured manner.

Second Normal Form (2NF)

A relation is in Second Normal Form if it is already in 1NF and all non-key attributes are fully dependent on the entire primary key. In the Online Shopping System, the ORDER_ITEM table uses a composite primary key consisting of order_id and product_id. Attributes such as quantity and

unit_price depend on the complete key rather than only a part of it. Additional entities such as PAYMENT, SHIPPING, REVIEW, and CART are separated into independent tables to eliminate partial dependencies.

Third Normal Form (3NF)

A relation is in Third Normal Form if it is already in 2NF and contains no transitive dependencies. In the proposed database, customer information depends only on customer_id, category information depends only on category_id, payment details depend only on payment_id, and shipping details depend only on shipping_id. By separating these entities into individual tables and linking them through foreign keys, transitive dependencies are eliminated.

UNF → 1NF → 2NF → 3NF

Table 3.6: Normalization Steps

Normal Form	Condition Satisfied	Anomaly Removed	Resulting Tables
UNF	Repeating groups exist	None	ORDER_DETAILS
1NF	Atomic values, no repeating groups	Repeating-group anomaly	CUSTOMER, CATEGORY, PRODUCT, ORDER, ORDER_ITEM
2NF	Full functional dependency on primary key	Partial dependency anomaly	PAYMENT, SHIPPING, REVIEW, CART separated
3NF	No transitive dependencies	Update, insertion, and deletion anomalies	Final normalized schema with 9 tables

CHAPTER 4: DATABASE IMPLEMENTATION

(Deliverables of Review – II)

4.1 Database and Table Creation – DDL

The database was created based on the relational schema and normalized design developed in the previous chapters. All tables were implemented with appropriate primary key, foreign key, unique, not null, check, and default constraints to ensure data integrity and consistency.

```
-- CREATE DATABASE
CREATE DATABASE OnlineShoppingSystem;
USE OnlineShoppingSystem;

-- CATEGORY TABLE
CREATE TABLE Category (
    category_id INT PRIMARY KEY AUTO_INCREMENT,
    category_name VARCHAR(100) NOT NULL UNIQUE,
    description VARCHAR(255)
);

-- CUSTOMER TABLE
CREATE TABLE Customer (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(15) NOT NULL,
    password VARCHAR(100) NOT NULL,
    address VARCHAR(255) NOT NULL,
    city VARCHAR(50),
    pincode VARCHAR(10)
);

-- PRODUCT TABLE
CREATE TABLE Product (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    description VARCHAR(255),
    price DECIMAL(10,2) NOT NULL CHECK(price > 0),
    stock_qty INT NOT NULL DEFAULT 0 CHECK(stock_qty >= 0),
    category_id INT NOT NULL,
    FOREIGN KEY (category_id)
        REFERENCES Category(category_id)
);

-- CART TABLE
CREATE TABLE Cart (
```

```

    cart_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT UNIQUE,
    created_date DATE DEFAULT (CURRENT_DATE),
    FOREIGN KEY (customer_id)
        REFERENCES Customer(customer_id)
);

-- ORDERS TABLE
CREATE TABLE Orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    order_date DATE DEFAULT (CURRENT_DATE),
    total_amount DECIMAL(10,2) DEFAULT 0,
    status VARCHAR(30) DEFAULT 'Pending',
    customer_id INT NOT NULL,
    FOREIGN KEY (customer_id)
        REFERENCES Customer(customer_id)
);

-- ORDER_ITEM TABLE
CREATE TABLE Order_Item (
    order_id INT,
    product_id INT,
    quantity INT NOT NULL CHECK(quantity > 0),
    unit_price DECIMAL(10,2) NOT NULL CHECK(unit_price > 0),

    PRIMARY KEY(order_id, product_id),

    FOREIGN KEY(order_id)
        REFERENCES Orders(order_id)
        ON DELETE CASCADE,

    FOREIGN KEY(product_id)
        REFERENCES Product(product_id)
        ON DELETE RESTRICT
);

-- PAYMENT TABLE
CREATE TABLE Payment (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT UNIQUE,
    payment_mode VARCHAR(30) NOT NULL,
    amount DECIMAL(10,2) NOT NULL CHECK(amount > 0),
    pay_date DATE,
    status VARCHAR(20) DEFAULT 'Pending',

    FOREIGN KEY(order_id)
        REFERENCES Orders(order_id)

```

```
);

-- SHIPPING TABLE
CREATE TABLE Shipping (
    shipping_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT UNIQUE,
    address VARCHAR(255) NOT NULL,
    courier VARCHAR(100),
    tracking_no VARCHAR(100) UNIQUE,
    delivery_date DATE,

    FOREIGN KEY(order_id)
        REFERENCES Orders(order_id)
);

-- REVIEW TABLE
CREATE TABLE Review (
    review_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    product_id INT NOT NULL,
    rating INT CHECK(rating BETWEEN 1 AND 5),
    comment VARCHAR(255),
    review_date DATE DEFAULT (CURRENT_DATE),

    FOREIGN KEY(customer_id)
        REFERENCES Customer(customer_id),

    FOREIGN KEY(product_id)
        REFERENCES Product(product_id)
);

-- ALTER TABLE EXAMPLE
ALTER TABLE Product
ADD COLUMN brand VARCHAR(50);
```



Fig. 4.1: Successful Creation of Tables

4.2 Sample Data Insertion – DML

Table 4.1: Sample Data Summary

S.No	Table Name	No. of Records Inserted	Screenshot Reference
1	CATEGORY	5	Fig. 4.2(a)
2	CUSTOMER	10	Fig. 4.2(b)
3	PRODUCT	15	Fig. 4.2(c)
4	CART	10	Fig. 4.2(d)
5	ORDERS	12	Fig. 4.2(e)
6	ORDER_ITEM	25	Fig. 4.2(f)
7	PAYMENT	12	Fig. 4.2(g)
8	SHIPPING	12	Fig. 4.2(h)
9	REVIEW	20	Fig. 4.2(i)

```

-- CATEGORY
INSERT INTO Category(category_name, description)
VALUES
('Electronics','Electronic Devices'),
('Fashion','Clothing and Accessories'),
('Books','Educational and Story Books'),
('Home Appliances','Home Utility Products'),
('Sports','Sports Equipment');

-- CUSTOMER
INSERT INTO Customer
(name,email,phone,password,address,city,pincode)
VALUES
('Rahul Kumar','rahul@gmail.com','9876543210','rahul123',
'Hyderabad','Hyderabad','500001'),

('Priya Sharma','priya@gmail.com','9876543211','priya123',
'Vijayawada','Vijayawada','520001'),

('Anil Kumar','anil@gmail.com','9876543212','anil123',
'Visakhapatnam','Vizag','530001');

-- PRODUCT
INSERT INTO Product
(name,description,price,stock_qty,category_id,brand)
VALUES
('Laptop','Dell Inspiron',55000,20,1,'Dell'),
('Smartphone','Samsung Galaxy',25000,50,1,'Samsung'),
('T-Shirt','Cotton T-Shirt',799,100,2,'Puma'),
('Database Concepts','DBMS Textbook',650,40,3,'McGraw Hill');

-- CART
INSERT INTO Cart(customer_id)
VALUES (1),(2),(3);

-- ORDERS
INSERT INTO Orders
(total_amount,status,customer_id)
VALUES
(55799,'Confirmed',1),
(25000,'Delivered',2),
(650,'Pending',3);

-- ORDER_ITEM
INSERT INTO Order_Item
(order_id,product_id,quantity,unit_price)

```

```

VALUES
(1,1,1,55000),
(1,3,1,799),
(2,2,1,25000),
(3,4,1,650);

-- PAYMENT
INSERT INTO Payment
(order_id,payment_mode,amount,pay_date,status)
VALUES
(1,'UPI',55799,'2025-09-01','Completed'),
(2,'Credit Card',25000,'2025-09-02','Completed'),
(3,'Cash on Delivery',650,'2025-09-03','Pending');

-- SHIPPING
INSERT INTO Shipping
(order_id,address,courier,tracking_no,delivery_date)
VALUES
(1,'Hyderabad','BlueDart','TRK1001','2025-09-05'),
(2,'Vijayawada','DTDC','TRK1002','2025-09-06'),
(3,'Visakhapatnam','Delhivery','TRK1003','2025-09-08');

-- REVIEW
INSERT INTO Review
(customer_id,product_id,rating,comment)
VALUES
(1,1,5,'Excellent Laptop'),
(2,2,4,'Good Performance'),
(3,4,5,'Very Useful Book');

```

Fig. 4.2(a): Sample Data Inserted into Category Table

Results Copy as Markdown

Query #1 Execution time: 0.3ms

category_id	category_name	description
1	Electronics	Electronic Devices
2	Fashion	Clothing and Accessories
3	Books	Educational and Story Books
4	Home Appliances	Home Utility Products
5	Sports	Sports Equipment

Fig. 4.2(b): Sample Data Inserted into Customer Table

Results

Copy as Markdown

Query #1 Execution time: 0.22ms

customer_id	name	email	phone	password	address	city	pincode
1	Rahul Kumar	rahul@gmail.com	9876543210	rahul123	Hyderabad	Hyderabad	500001
2	Priya Sharma	priya@gmail.com	9876543211	priya123	Vijayawada	Vijayawada	520001
3	Anil Kumar	anil@gmail.com	9876543212	anil123	Visakhapatnam	Vizag	530001

Fig. 4.2(c): Sample Data Inserted into Product Table

Results

Copy as Markdown

Query #1 Execution time: 0.23ms

product_id	name	description	price	stock_qty	category_id	brand
1	Laptop	Dell Inspiron	55000.0	20	1	Dell
2	Smartphone	Samsung Galaxy	25000.0	50	1	Samsung
3	T-Shirt	Cotton T-Shirt	799.0	100	2	Puma
4	Database Concepts	DBMS Textbook	650.0	40	3	McGraw Hill

Fig. 4.2(d): Sample Data Inserted into Cart Table

Results

Copy as Markdown

Query #1

Execution time: 0.28ms

cart_id	customer_id	created_date
1	1	2026-09-26
2	2	2026-09-26
3	3	2026-09-26

Fig. 4.2(e): Sample Data Inserted into Orders Table

Results

Copy as Markdown

Query #1 Execution time: 0.21ms

order_id	order_date	total_amount	status	customer_id
1	2026-09-26	55799.0	Confirmed	1
2	2026-09-26	25000.0	Delivered	2
3	2026-09-26	650.0	Pending	3

Fig. 4.2(f): Sample Data Inserted into Order_Item Table

Results

Copy as Markdown

Query #1 Execution time: 0.21ms

order_id	product_id	quantity	unit_price
1	1	1	55000.0
1	3	1	799.0
2	2	1	25000.0
3	4	1	650.0

Fig. 4.2(g): Sample Data Inserted into Payment Table

Results

Copy as Markdown

Query #1 Execution time: 0.29ms

payment_id	order_id	payment_mode	amount	pay_date	status
1	1	UPI	55799.0	2025-09-01	Completed
2	2	Credit Card	25000.0	2025-09-02	Completed
3	3	Cash on Delivery	650.0	2025-09-03	Pending

Fig. 4.2(h): Sample Data Inserted into Shipping Table

Results

Copy as Markdown

Query #1 Execution time: 0.22ms

shipping_id	order_id	address	courier	tracking_no	delivery_date
1	1	Hyderabad	BlueDart	TRK1001	2025-09-05
2	2	Vijayawada	DTDC	TRK1002	2025-09-06
3	3	Visakhapatnam	Delhivery	TRK1003	2025-09-08

Fig. 4.2(i): Sample Data Inserted into Review Table

Results

Copy as Markdown

Query #1 Execution time: 0.21ms

review_id	customer_id	product_id	rating	comment	review_date
1	1	1	5	Excellent Laptop	2026-09-26
2	2	2	4	Good Performance	2026-09-26
3	3	4	5	Very Useful Book	2026-09-26

Fig. 4.2: Sample Data Inserted into the Tables

4.3 Constraints and Data Integrity

Table 4.2: Constraints and Data Integrity Rules

Constraint / Validation	Applied On (Table.Column)	Purpose
PRIMARY KEY	CUSTOMER.customer_id	Ensures each customer has a unique identifier
PRIMARY KEY	PRODUCT.product_id	Prevents duplicate product records
PRIMARY KEY (Composite)	ORDER_ITEM(order_id, product_id)	Prevents duplicate product entries in the same order
FOREIGN KEY	PRODUCT.category_id	Ensures every product belongs to a valid category
FOREIGN KEY	ORDERS.customer_id	Ensures every order is linked to an existing customer
FOREIGN KEY	PAYMENT.order_id	Ensures payment records exist only for valid orders
FOREIGN KEY	SHIPPING.order_id	Ensures shipping details correspond to valid orders
FOREIGN KEY	REVIEW.customer_id	Allows reviews only from registered customers
FOREIGN KEY	REVIEW.product_id	Allows reviews only for existing products
NOT NULL	CUSTOMER.name	Prevents empty customer names
NOT NULL	CUSTOMER.email	Ensures email information is mandatory
UNIQUE	CUSTOMER.email	Prevents duplicate customer accounts
UNIQUE	CATEGORY.category_name	Prevents duplicate category names
CHECK(price > 0)	PRODUCT.price	Prevents invalid product prices
CHECK(stock_qty >= 0)	PRODUCT.stock_qty	Prevents negative inventory values

CHECK(quantity > 0)	ORDER_ITEM.quantity	Ensures valid order quantities
CHECK(rating BETWEEN 1 AND 5)	REVIEW.rating	Restricts ratings to 1–5 stars
DEFAULT 'Pending'	ORDERS.status	Assigns default order status
DEFAULT CURRENT_DATE	ORDERS.order_date	Automatically stores order date
ON DELETE CASCADE	ORDER_ITEM.order_id	Deletes order items when an order is removed
ON DELETE RESTRICT	ORDER_ITEM.product_id	Prevents deletion of products referenced in orders

4.4 Data Validation Rules

To confirm that the constraints actually work, three invalid statements were executed deliberately. In each case, the database rejected the statement and no row was inserted or updated.

(a) Referential Integrity — an order for a customer who does not exist:

```
INSERT INTO Orders
```

```
(order_id, order_date, total_amount, status, customer_id)
```

```
VALUES
```

```
(101, '2025-09-15', 5000, 'Pending', 999);
```

ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails

(OnlineShoppingSystem.Orders , CONSTRAINT orders_ibfk_1, FOREIGN KEY (customer_id)

REFERENCES Customer(customer_id))

(b) CHECK Constraint — a product with a negative price:

```
INSERT INTO Product
```

```
(product_id, name, description, price, stock_qty, category_id)
```

```
VALUES
```

```
(20, 'Test Product', 'Invalid Product', -500, 10, 1);
```

ERROR 3819 (HY000): Check constraint 'product_chk_1' is violated.

(c) UNIQUE Constraint — duplicate customer email:

```
INSERT INTO Customer
```

(name, email, phone, password, address, city, pincode)

VALUES

('Test User', 'rahul@gmail.com', '9876543299', 'test123', 'Hyderabad', 'Hyderabad', '500001');

ERROR 1062 (23000): Duplicate entry 'rahul@gmail.com' for key 'Customer.email'.

Fig. 4.3: Referential Integrity Enforced by the Database



CHAPTER 5: SQL OPERATIONS AND QUERIES

5.1 DQL – Basic Retrieval Queries

Q1. Display all customer details

SQL:

```
SELECT * FROM Customer;
```

OUTPUT:

Results						Copy as Markdown
Query #1 Execution time: 0.33ms						
customer_id	name	order_id	order_date	total_amount	status	
1	Rahul Kumar	1	2026-09-26	55799.0	Confirmed	
2	Priya Sharma	2	2026-09-26	25000.0	Delivered	
3	Anil Kumar	3	2026-09-26	650.0	Pending	

Q2. Display products whose price is greater than 10,000

SQL:

```
SELECT * FROM Product  
WHERE price > 10000;
```

OUTPUT:

Results							Copy as Markdown
Query #1 Execution time: 0.26ms							
product_id	name	description	price	stock_qty	category_id	brand	
1	Laptop	Dell Inspiron	55000.0	20	1	Dell	
2	Smartphone	Samsung Galaxy	25000.0	50	1	Samsung	

Q3. Display only delivered orders

SQL:

```
SELECT * FROM Orders  
WHERE status='Delivered';
```

OUTPUT:

Results					Copy as Markdown
Query #1					Execution time: 0.22ms
order_id	order_date	total_amount	status	customer_id	
2	2026-09-26	25000.0	Delivered	2	

Table 5.1: Basic Retrieval Queries

5.2 Join Operations

Q4. Natural join

SQL:

```
SELECT customer_id, name, order_id, order_date, total_amount
FROM Customer NATURAL JOIN Orders;
```

OUTPUT:

Results					Copy as Markdown
Query #1					Execution time: 0.29ms
customer_id	name	order_id	order_date	total_amount	
1	Rahul Kumar	1	2026-09-26	55799.0	
2	Priya Sharma	2	2026-09-26	25000.0	
3	Anil Kumar	3	2026-09-26	650.0	

Q5. Inner join

SQL:

```
SELECT C.name, O.order_id, O.total_amount
FROM Customer C
INNER JOIN Orders O
ON C.customer_id = O.customer_id;
```

OUTPUT:

Results			Copy as Markdown
Query #1	Execution time: 0.25ms		
name	order_id	total_amount	
Rahul Kumar	1	55799.0	
Priya Sharma	2	25000.0	
Anil Kumar	3	650.0	

Q6. Left outer join

SQL:

```
SELECT C.customer_id, C.name, O.order_id
FROM Customer C
LEFT OUTER JOIN Orders O
ON C.customer_id = O.customer_id
```

OUTPUT:

Results			Copy as Markdown
Query #1	Execution time: 0.25ms		
customer_id	name	order_id	
1	Rahul Kumar	1	
2	Priya Sharma	2	
3	Anil Kumar	3	

Q7. Right outer join

SQL:

```
SELECT C.name, O.order_id, O.status
FROM Customer C
RIGHT OUTER JOIN Orders O
ON C.customer_id = O.customer_id;
```

OUTPUT:

Results			Copy as Markdown
Query #1	Execution time: 0.4ms		
name	order_id	status	
Rahul Kumar	1	Confirmed	
Priya Sharma	2	Delivered	
Anil Kumar	3	Pending	

5.3 Aggregate Functions, GROUP BY and HAVING

Q8. SUM() –Calculates total revenue generated from all orders

SQL:

```
SELECT SUM(total_amount) AS Total_Sales
FROM Orders;
```

OUTPUT:

Results		Copy as Markdown
Query #1	Execution time: 0.25ms	
Total_Sales		
81449.0		

Q9. GROUP BY—Shows how many orders each customer has placed

SQL:

```
SELECT customer_id,
       COUNT(order_id) AS Total_Orders
FROM Orders
GROUP BY customer_id;
```

OUTPUT:

Results		Copy as Markdown
Query #1	Execution time: 0.26ms	
customer_id	Total_Orders	
1	1	
2	1	
3	1	

Q10. GROUP BY with HAVING—Displays categories containing more than one product

SQL:

```
SELECT category_id,
       COUNT(product_id) AS Total_Products
FROM Product
GROUP BY category_id
```

```
HAVING COUNT(product_id) > 1;
```

OUTPUT:

Results [Copy as Markdown](#)

Query #1 [Execution time: 0.29ms](#)

category_id	Total_Products
1	2

Q10. GROUP BY with HAVING

SQL:

```
SELECT customer_id,  
       COUNT(order_id) AS Total_Orders  
FROM Orders  
GROUP BY customer_id  
HAVING COUNT(order_id) > 1;
```

OUTPUT:

Results [Copy as Markdown](#)

Query #1 [Execution time: 0.29ms](#)

customer_id	Total_Orders
-------------	--------------

There are no results to be displayed.

5.4 Nested and Correlated Subqueries

Q11. Nested Subquery—Products whose price is above the average product price

SQL:

```
SELECT *  
FROM Product  
WHERE price >  
(  
  SELECT AVG(price)  
  FROM Product  
);
```

OUTPUT:

Results

[Copy as Markdown](#)

Query #1 Execution time: 0.44ms

product_id	name	description	price	stock_qty	category_id	brand
1	Laptop	Dell Inspiron	55000.0	20	1	Dell
2	Smartphone	Samsung Galaxy	25000.0	50	1	Samsung

Q12. Nested Subquery—Displays customers who have placed at least one order

SQL:

```
SELECT *
FROM Customer
WHERE customer_id IN
(
    SELECT customer_id
    FROM Orders
);
```

OUTPUT:

Results

[Copy as Markdown](#)

Query #1 Execution time: 0.35ms

customer_id	name	email	phone	password	address	city	pincode
1	Rahul Kumar	rahul@gmail.com	9876543210	rahul123	Hyderabad	Hyderabad	500001
2	Priya Sharma	priya@gmail.com	9876543211	priya123	Vijayawada	Vijayawada	520001
3	Anil Kumar	anil@gmail.com	9876543212	anil123	Visakhapatnam	Vizag	530001

Q13. Correlated Sub query –Order amount is greater than their own average order value

SQL:

```
SELECT DISTINCT C.customer_id,
               C.name
FROM Customer C
JOIN Orders O
ON C.customer_id = O.customer_id
WHERE O.total_amount >
(
    SELECT AVG(O2.total_amount)
    FROM Orders O2
    WHERE O2.customer_id = O.customer_id
);
```

OUTPUT:

Results							Copy as Markdown
Query #1 Execution time: 0.43ms							
product_id	name	description	price	stock_qty	category_id	brand	
1	Laptop	Dell Inspiron	55000.0	20	1	Dell	
2	Smartphone	Samsung Galaxy	25000.0	50	1	Samsung	

Q14. Correlated Subquery—Displays products whose price is greater than average price

SQL:

```
SELECT P1.product_id,
       P1.name,
       P1.price
FROM Product P1
WHERE P1.price >
(
  SELECT AVG(P2.price)
  FROM Product P2
  WHERE P2.category_id = P1.category_id
);
```

OUTPUT:

Results			Copy as Markdown
Query #1 Execution time: 0.42ms			
product_id	name	price	
1	Laptop	55000.0	

5.5 Views and Relational Set Operations

Q15. UNION—Returns unique customer IDs appearing in either table

SQL:

```
SELECT customer_id
FROM Customer

UNION
```

```
SELECT customer_id
FROM Orders
```

OUTPUT:

Results

Copy as Markdown

Query #1 Execution time: 0.32ms

customer_id
3
2
1

Q16. UNION ALL—Returns all customer ids including duplicate values

SQL:

```
SELECT customer_id
FROM Customer
```

```
UNION ALL
```

```
SELECT customer_id
FROM Orders;
```

OUTPUT:

Results

Copy as Markdown

Query #1 Execution time: 0.31ms

customer_id
3
2
1
1
2
3

Q17. INTERSECT Equivalent—Displays customers who have placed orders

SQL:

```
SELECT customer_id
FROM Customer
WHERE customer_id IN
(
    SELECT customer_id
```

```

        FROM Orders
    );

```

OUTPUT:

Results

Copy as Markdown

Query #1 Execution time: 0.26ms

customer_id
3
2
1

5.6 DCL and TCL Operations

Data Control Language commands were used to create a read-only user for the examination section, and Transaction Control Language commands were used to demonstrate that a partially completed update can be reversed.

(a) DCL—GRANT and REVOKE

```

GRANT SELECT, INSERT, UPDATE
ON Product
TO user1;
REVOKE UPDATE
ON Product
FROM user1;

```

(b) TCL—COMMIT, ROLLBACK AND SAVEPOINT

```

INSERT INTO Product(product_id, name, price, stock)
VALUES (101, 'Laptop', 50000, 20);

```

```

COMMIT;
DELETE FROM Product
WHERE product_id = 101;

```

```

ROLLBACK;
SAVEPOINT sp1;

```

```

UPDATE Product
SET price = 55000
WHERE product_id = 101;
ROLLBACK TO sp1;

```

CHAPTER 6: MODULES AND FUNCTIONALITIES

Table 6.1: Modules and Their Functionalities

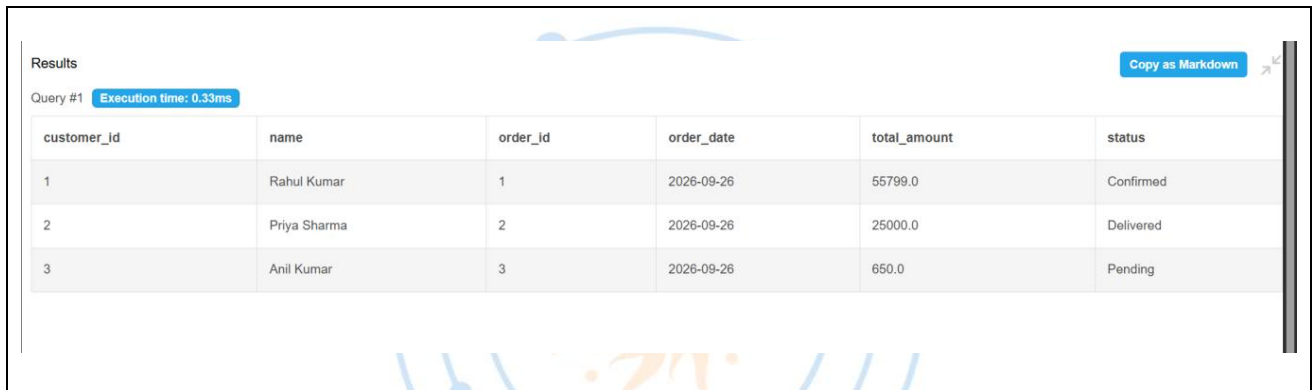
S.No	Module Name	Functionality	Tables Involved
1	Customer Management Module	Manages customer registration, login, profile information, addresses, and account details.	CUSTOMER
2	Category Management Module	Stores and manages product categories such as Electronics, Fashion, Books, etc.	CATEGORY
3	Product Management Module	Handles product information including product name, description, price, stock quantity, and category assignment	PRODUCT, CATEGORY
4	Shopping Cart Module	Allows customers to add, view, update, and remove products before placing an order.	CART, PRODUCT, CUSTOMER
5	Order Management Module	Processes customer orders, maintains order details, order status, and order history.	ORDERS, ORDER_ITEM, CUSTOMER
6	Payment Management Module	Records payment transactions, payment methods, payment status, and payment dates.	PAYMENT, ORDERS
7	Shipping Management Module	Stores shipping details, courier information, tracking numbers, and delivery dates.	SHIPPING, ORDERS
8	Review and Rating Module	Allows customers to submit ratings and reviews for purchased products.	REVIEW, CUSTOMER, PRODUCT
8	Inventory Management Module	Maintains available stock quantities and updates inventory after purchases.	PRODUCT, ORDER_ITEM
9	Reporting and Query Module	Generates reports such as customer orders, sales details, product information, payment summaries, and shipping reports using SQL queries.	CUSTOMER, PRODUCT, ORDERS, PAYMENT, SHIPPING, REVIEW

The Online Shopping System is centered around the Order Management module, which connects customers, products, payments, and shipping information through foreign key relationships. The ORDER_ITEM table resolves the many-to-many relationship between orders and products, while the integrated design ensures data integrity, reduces redundancy, and supports efficient transaction processing across the database

CHAPTER 7: RESULTS – SAMPLE OUTPUTS AND SCREENSHOTS

Report 1: Customer order report

```
SELECT C.customer_id,  
       C.name,  
       O.order_id,  
       O.order_date,  
       O.total_amount,  
       O.status  
FROM Customer C  
JOIN Orders O  
ON C.customer_id = O.customer_id;
```



The screenshot shows a database query results interface. At the top, it says 'Results' and 'Query #1' with an execution time of '0.33ms'. There is a 'Copy as Markdown' button. Below this is a table with 6 columns: customer_id, name, order_id, order_date, total_amount, and status. The table contains 3 rows of data.

customer_id	name	order_id	order_date	total_amount	status
1	Rahul Kumar	1	2026-09-26	55799.0	Confirmed
2	Priya Sharma	2	2026-09-26	25000.0	Delivered
3	Anil Kumar	3	2026-09-26	650.0	Pending

Fig. 7.1: Shows customer order history

Report 2: product inventory report

```
SELECT product_id,  
       name,  
       price,  
       stock_qty  
FROM Product  
ORDER BY stock_qty ASC
```

--

Results				Copy as Markdown
Query #1 Execution time: 0.24ms				
product_id	name	price	stock_qty	
1	Laptop	55000.0	20	
4	Database Concepts	650.0	40	
2	Smartphone	25000.0	50	
3	T-Shirt	799.0	100	

Fig. 7.2: Product inventory report

Report 3: Sales summary report

```
SELECT COUNT(order_id) AS Total_Orders,
       SUM(total_amount) AS Total_Sales,
       AVG(total_amount) AS Average_Order_Value
FROM Orders;
```

Results			Copy as Markdown
Query #1 Execution time: 0.27ms			
Total_Orders	Total_Sales	Average_Order_Value	
3	81449.0	27149.666667	

Fig. 7.3: Sales summary report

REPORT 4: Payment and shipping report

```
SELECT O.order_id,
       C.name,
       P.payment_mode,
       P.status AS Payment_Status,
       S.courier,
       S.tracking_no
FROM Orders O
```

JOIN Customer C

ON O.customer_id = C.customer_id

JOIN Payment P

ON O.order_id = P.order_id

JOIN Shipping S

ON O.order_id = S.order_id;

Results

Query #1 Execution time: 0.3ms

Copy as Markdown

order_id	name	payment_mode	Payment_Status	courier	tracking_no
1	Rahul Kumar	UPI	Completed	BlueDart	TRK1001
2	Priya Sharma	Credit Card	Completed	DTDC	TRK1002
3	Anil Kumar	Cash on Delivery	Pending	Delhivery	TRK1003

Fig. 7.4: payment and shipping report

CHAPTER 8: TESTING AND DATA VALIDATION

The database was tested using both valid and invalid inputs to verify the correctness of constraints, relationships, and SQL operations. All test cases produced the expected results, confirming that the Online Shopping System maintains data integrity and functions correctly.

Table 8.1: Test Cases and Results

TC No.	Test Case Description	Input / Action	Expected Result	Actual Result	Status
TC01	Insert a valid customer record	Valid customer details	Record inserted successfully	Record inserted	Pass
TC02	Insert customer with duplicate email	Existing email address	Database rejects insertion	Error generated	Pass
TC03	Insert product with negative price	Price = -500	CHECK constraint violation	Error generated	Pass
TC04	Insert order with invalid customer ID	customer_id = 999	Foreign key violation	Error generated	Pass
TC05	Insert review with rating > 5	Rating = 7	CHECK constraint violation	Error generated	Pass
TC06	Retrieve customer order details	Execute JOIN query	Correct records displayed	Correct output displayed	Pass
TC07	Generate sales summary report	Execute aggregate query	Total sales displayed	Correct output displayed	Pass
TC08	Delete order record	Delete valid order	Order removed successfully	Order removed	Pass

CHAPTER 9: GITHUB REPOSITORY AND TEAM CONTRIBUTION

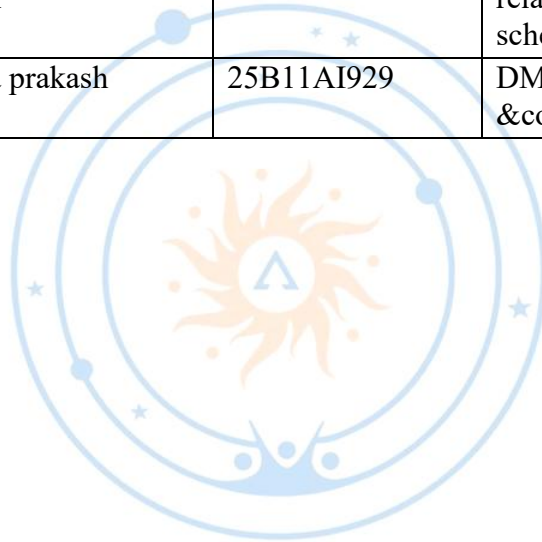
9.1 Repository Details

GitHub Repository Link: <https://github.com/panganikilsaikumarreddy2507-sketch/online-shopping-system>

9.2 Team Responsibilities and Contribution

Table 9.1: Team Responsibilities

S.No	Student Name	Roll Number	Role / Responsibility
1.	P.Nikhil sai kumar reddy	25B11AI895	Database design and SQL integration
2.	P.Srilekha thara	25B11AI966	Documentation ,queries and testing
3.	A.Shanmukha priya lakshmi	25B11AI056	ER Model and relational database schema diagram
4.	P.Surya prakash	25B11AI929	DML,Tables ,keys &constraints



CHAPTER 10: CONCLUSION AND FUTURE ENHANCEMENTS

10.1 Conclusion

The Online Shopping System database was successfully designed and implemented using MySQL to manage customers, products, categories, shopping carts, orders, payments, shipping details, and reviews. The database was normalized up to Third Normal Form (3NF) to reduce redundancy and improve data consistency. Various SQL operations such as DDL, DML, DQL, joins, aggregate functions, subqueries, views, and reports were successfully executed. The implementation of primary keys, foreign keys, and validation constraints ensured data integrity, reliability, and efficient data management. The project demonstrates the practical application of database management concepts in developing a structured and scalable e-commerce database system.

10.2 Limitations of the Present Work

Although the database successfully manages online shopping operations, it has certain limitations. The project focuses only on the database layer and does not include a web or mobile user interface. Real-time payment gateway integration, advanced security features, and automated inventory tracking are not implemented. The system also uses sample data and does not address large-scale performance optimization or concurrent user access that may occur in real-world e-commerce applications.

10.3 Future Enhancements

The Online Shopping System can be enhanced by integrating a web-based or mobile application interface for customer interaction. Additional features such as inventory management, order tracking, product recommendation systems, discount and coupon management, and customer support modules can be added. Integration with online payment gateways, advanced authentication mechanisms, and data analytics tools can further improve functionality, security, and user experience. These enhancements would make the system more suitable for deployment in real-world e-commerce environments.

REFERENCES

List textbooks, reference books, documentation and websites referred to. Follow the format shown below.

6. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, “Database System Concepts”, 8th Edition, McGraw-Hill, ISBN: 978-1260230508.
7. Raghurama Krishnan and Johannes Gehrke, “Database Management Systems”, 4th Edition, McGraw-Hill, ISBN: 978-1260091195.
8. Ramez Elmasri and Shamkant B. Navathe, “Fundamentals of Database Systems”, 7th Edition, Pearson, ISBN: 978-0137504277.
9. _____
10. _____

